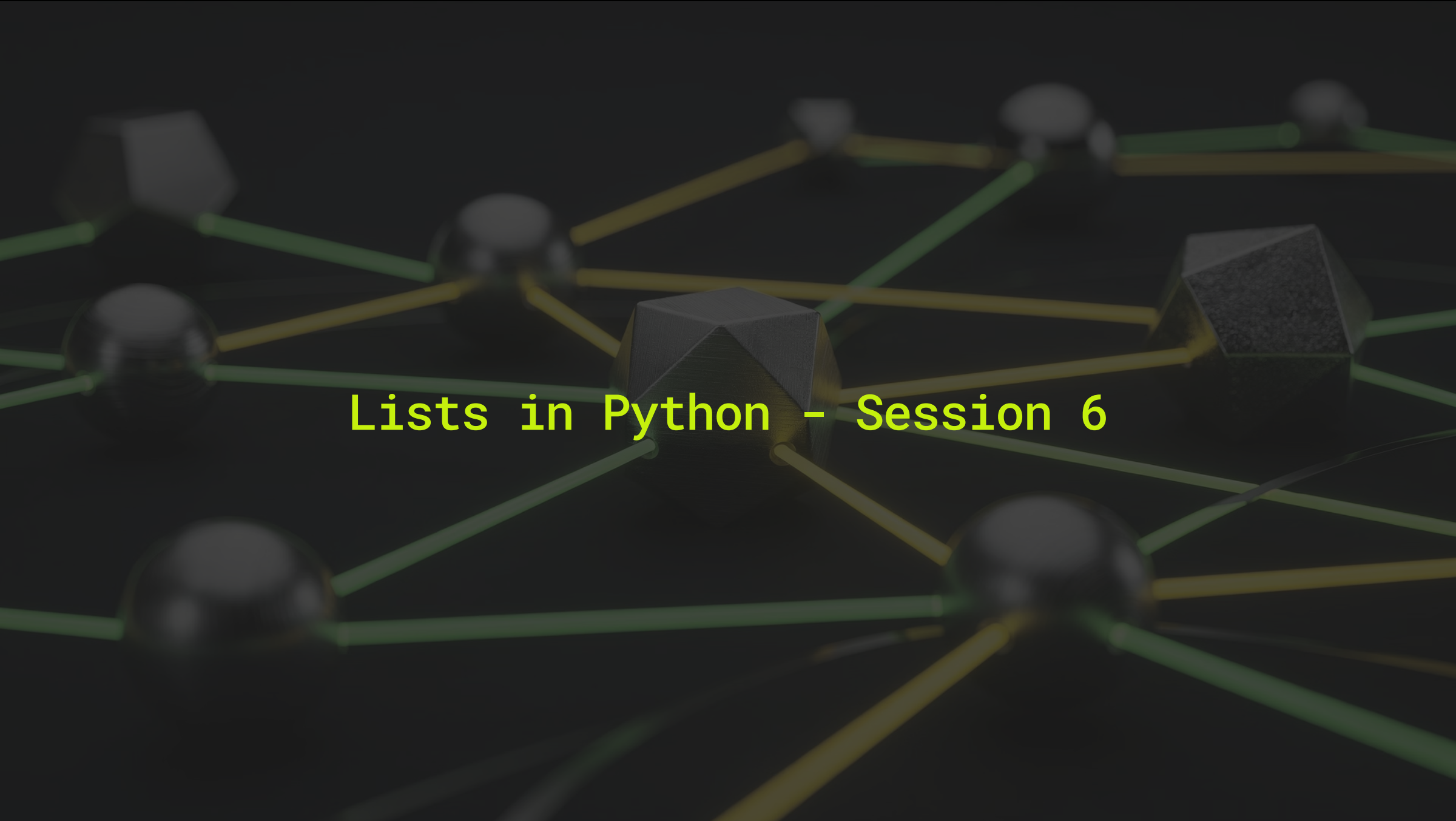




Lists in Python – Session 6

What is a List in Python?

So far, a variable stores **one value at a time**. But what if you need to store the marks of 5 students, or the names of 10 cities? Creating a separate variable for each one quickly becomes messy and impractical. A **list** solves this — it is a collection that stores **multiple values in a single variable**, kept in a specific order.

Without a List – Messy 😞

```
student1 = "Rahul"  
student2 = "Priya"  
student3 = "Karan"
```

Three variables for three names. Imagine doing this for 100 students!

With a List – Clean ✅

```
students = ["Rahul", "Priya", "Karan"]
```

All three names stored neatly in **one variable**. Easy to manage, scale, and read.

Key Properties of a List



Ordered

Items stay in the exact order you put them in. The first item is always first.



Mutable

You can change, add, or remove items even after the list is created.



Allows Duplicates

The same value can appear more than once — lists do not enforce uniqueness.



Any Data Type

A single list can hold integers, strings, booleans, or even other lists!



Think of a list like a **row of labelled boxes** — each box holds one value, they are arranged in order, and you can swap, add, or remove items whenever you need to.

Creating a List

Lists in Python are defined by enclosing comma-separated items within square brackets `[]`. You can create lists with various data types, an empty list, or even lists containing duplicate values. Let's see some common ways to initialise them.

Empty List

Start with an empty list and add items later:

```
my_list = []
```

Numbers (Integers/Floats)

Store a sequence of numeric values:

```
marks = [85, 90, 78, 92, 88]
```

Text (Strings)

Hold a collection of words or phrases:

```
cities = ["Delhi", "Mumbai",  
"Pune", "Chennai"]
```

Mixed Data Types

Lists can contain different types of data in one go:

```
profile = ["Rahul", 21, 9.2, True]
```

Lists with Duplicates

Python lists readily allow repeated values:

```
scores = [10, 20, 10, 30, 20, 10]
```

Checking Type and Length

Use `type()` to confirm it's a list, and `len()` to get the item count:

```
print(type(marks)) # <class  
'list'>  
print(len(marks)) # 5
```

 A list is incredibly versatile – it can contain any data type, and even a mix of types in the same list. Each value stored within a list is referred to as an **element** or **item**.

List Operators: Joining and Repeating

Python offers simple operators to manipulate lists efficiently. The addition (`+`) operator allows you to combine lists, while the multiplication (`*`) operator helps in repeating list elements.

Joining Lists with `+`

The `+` operator concatenates (joins) two or more lists, creating an entirely new list. The original lists remain unaffected.

```
list_a = [10, 20]
list_b = [30, 40, 50]
combined = list_a + list_b
print(combined)
# Output: [10, 20, 30, 40, 50]
```

¹ Repeating Lists with `*`

The `*` operator repeats the elements of a list a specified number of times, returning a new list. The multiplier must be an integer.

```
items = ['apple', 'banana']
repeated = items * 2
print(repeated)
# Output: ['apple', 'banana', 'apple', 'banana']
```

 These operations are particularly useful for preparing data or generating sequences without writing complex loops.

Built-in Functions on Lists

Python offers a suite of powerful built-in functions that operate directly on lists, allowing you to perform common operations efficiently without writing manual loops. These functions are highly optimised for performance and make your code more readable.

len() - Get Length

Returns the number of elements present in the list.

```
marks = [85, 92, 78, 95, 60, 88]
print(len(marks)) # Output: 6
```

min() - Smallest Value

Returns the smallest item from a list of numbers.

```
marks = [85, 92, 78, 95, 60, 88]
print(min(marks)) # Output: 60
```

max() - Largest Value

Returns the largest item from a list of numbers.

```
marks = [85, 92, 78, 95, 60, 88]
print(max(marks)) # Output: 95
```

sum() - Sum of Elements

Returns the sum of all numeric elements in the list.

```
marks = [85, 92, 78, 95, 60, 88]
print(sum(marks)) # Output: 498
```

sorted() - Create Sorted List

Returns a new list containing all items from the iterable in ascending order. The original list remains unchanged.

```
marks = [85, 92, 78, 95, 60, 88]
print(sorted(marks))          # Output: [60, 78, 85, 88, 92, 95]
print(sorted(marks, reverse=True)) # Output: [95, 92, 88, 85, 78, 60]
print(marks)                  # Output: [85, 92, 78, 95, 60, 88]
```

Practical Use: Calculating Average Marks

Combining `sum()` and `len()` provides a simple way to calculate the average of numerical data in a list.

```
marks = [85, 92, 78, 95, 60, 88]
average = sum(marks) / len(marks)
print(f"Average: {average:.2f}") # Output: Average: 83.00
```

Accessing List Elements: Indexing

Every element in a Python list occupies a specific position, known as its **index**. It's crucial to remember that Python, like many programming languages, uses **zero-based indexing**. This means the first element is at index 0, the second at 1, and so forth.

Positive Indexing: Left to Right

Use positive indices to access elements from the beginning of the list, moving towards the right. The first element is always at index 0.

```
fruits = ["apple", "mango", "banana", "grapes", "orange"]
#      0      1      2      3      4

print(fruits[0]) # Output: apple
print(fruits[2]) # Output: banana
print(fruits[4]) # Output: orange
```

Negative Indexing: Right to Left

Negative indices allow you to access elements from the end of the list, moving towards the left. The last element is at index -1.

```
fruits = ["apple", "mango", "banana", "grapes", "orange"]
#      -5      -4      -3      -2      -1

print(fruits[-1]) # Output: orange
print(fruits[-3]) # Output: banana
print(fruits[-5]) # Output: apple
```

Updating Elements Using Index

You can modify a list element by assigning a new value to its specific index. This directly changes the item at that position.

```
fruits = ["apple", "mango", "banana", "grapes", "orange"]
fruits[1] = "kiwi" # Replaces "mango" with "kiwi"

print(fruits) # Output: ["apple", "kiwi", "banana", "grapes", "orange"]
```

⚠ Watch Out: IndexError

Attempting to access an index that does not exist in the list will result in an **IndexError**, causing your program to crash. If a list has **N** elements, valid positive indices range from **0** to **N-1**, and valid negative indices from **-N** to **-1**.

Practice Questions: List Indexing

Now that you understand the basics of list indexing, put your knowledge to the test! Attempt these questions to solidify your understanding.

Question 49: Favourite Movies

Create a list of 5 of your favourite movies. Print the first, last, and middle movie from your list using both positive and negative indexing where appropriate.

```
# Example Output (using hypothetical movie list):  
# First movie: The Shawshank Redemption  
# Last movie: Forrest Gump  
# Middle movie: The Matrix
```

Question 50: Number Replacement

Create a list of 5 numbers (e.g., [10, 20, 30, 40, 50]). Replace the second and fourth elements of this list with the number 0 using indexing. Print the updated list.

```
# Expected Output (for the example list above):  
# [10, 0, 30, 0, 50]
```

📌 Pay close attention to zero-based indexing when selecting elements to replace or print.

Slicing Python Lists

Slicing is a powerful feature in Python that allows you to extract a specific portion or "slice" from a list. It doesn't modify the original list; instead, it returns a new list containing only the selected elements. This is incredibly useful for working with subsets of data efficiently.

Slicing Syntax

The general syntax for slicing is `list_name[start:stop:step]`:

Start Index

The index where the slice begins (inclusive). If omitted, it defaults to 0 (the beginning of the list).

Stop Index

The index where the slice ends (exclusive). The element at this index will **not** be included. If omitted, it defaults to the end of the list.

Step Value

Determines how many positions to jump between elements. If omitted, it defaults to 1. A negative step value reverses the order.

Examples of List Slicing

```
nums = [10, 20, 30, 40, 50, 60, 70, 80]
```

```
print(nums[1:5])  # Output: [20, 30, 40, 50] (Elements from index 1 up to (but not including) 5)
print(nums[:4])   # Output: [10, 20, 30, 40] (Starts from beginning, up to (not including) index 4)
print(nums[4:])   # Output: [50, 60, 70, 80] (Starts from index 4, goes to the end)
print(nums[::2])  # Output: [10, 30, 50, 70] (Starts from beginning, goes to end, steps by 2)
print(nums[1:7:2]) # Output: [20, 40, 60] (Starts at index 1, up to 7, steps by 2)
print(nums[::-1]) # Output: [80, 70, 60, 50, 40, 30, 20, 10] (Reverses the entire list)
```

Quick Tip: Reversing Lists

The slice `[::-1]` is the cleanest and most Pythonic way to reverse a list. Keep this handy, as it's a common and efficient trick!

Practice Questions: Built-in Functions

Apply your understanding of Python's built-in list functions with these practice exercises.

Question 51: Student Marks Analysis

Create a list of 6 hypothetical student marks (e.g., [75, 88, 62, 91, 70, 83]). Using built-in functions, print the highest mark, lowest mark, total sum of marks, and the average mark. Format the average to two decimal places.

```
# Expected Output (using hypothetical marks):  
# Highest Mark: 91  
# Lowest Mark: 62  
# Total Marks: 469  
# Average Mark: 78.17
```

Question 52: Sorted Lists

Create a list of numbers (e.g., [5, 2, 8, 1, 9, 3]). Print the list in ascending order and then in descending order using the `sorted()` function.

After printing, verify and print the original list to confirm it remains unchanged.

```
# Expected Output (using hypothetical list):  
# Ascending Order: [1, 2, 3, 5, 8, 9]  
# Descending Order: [9, 8, 5, 3, 2, 1]  
# Original List: [5, 2, 8, 1, 9, 3]
```

Looping Through a List

Iterating through the elements of a list is a fundamental operation in Python. It allows you to process each item, perform calculations, or display contents. Python provides elegant ways to achieve this using both `for` and `while` loops.

Using a `for` loop

The `for` loop is the most common and Pythonic way to iterate directly over the elements of a list. It automatically handles indexing and iteration until all items are processed.

```
pythonfruits = ["apple", "mango", "banana"]

for fruit in fruits:
    print(fruit)
```

Using a `while` loop

While less common for simple iteration, a `while` loop can also be used to traverse a list. This method often requires manually managing an index to access elements and control the loop's progression.

```
pythonfruits = ["apple", "mango", "banana"]
i = 0

while i < len(fruits):
    print(fruits[i])
    i += 1
```

 The `for` loop is generally preferred for iterating over sequences like lists due to its simplicity and readability. Use `while` loops when you need more control over the iteration process, such as when you might modify the list's length during iteration.

Looping with enumerate()

When you need to access both the index and the value of items while iterating through a list, Python's built-in `enumerate()` function is incredibly useful. It adds a counter to an iterable and returns it as an enumerate object. This object can then be used directly in for loops.

Basic Usage of enumerate()

```
fruits = ["apple", "mango", "banana"]
```

```
for index, fruit in enumerate(fruits):  
    print(f'{index}: {fruit}')
```

```
# Output:  
# 0: apple  
# 1: mango  
# 2: banana
```

In this example, `enumerate()` assigns an index starting from 0 to each element in the `fruits` list, allowing us to print both its position and its value.

enumerate() with Custom Start Index

You can specify a custom starting index for the counter using the `start` parameter, which is particularly helpful when you want the indexing to align with human-readable numbering (e.g., starting from 1 instead of 0).

```
fruits = ["apple", "mango", "banana"]
```

```
for index, fruit in enumerate(fruits, start=1):  
    print(f'{index}. {fruit}')
```

```
# Output:  
# 1. apple  
# 2. mango  
# 3. banana
```

Why Use enumerate()?

`enumerate()` makes your code cleaner and more efficient compared to manually tracking an index with a `while` loop or `range(len(list))`. It's the preferred Pythonic way when you require both index and value during iteration.

Practice Questions: Looping Through Lists

Test your proficiency in iterating over lists using `for` and `while` loops to solve common programming challenges.

Question 53: Find the Largest Element

Given a list of numbers, write Python code using a loop to find and print the largest element. Do not use the built-in `max()` function.

```
# Example:  
# numbers = [3, 1, 4, 1, 5, 9, 2, 6]  
# Expected Output: The largest element is: 9
```

Question 54: Check for Target Existence

Write a program that takes a list and a target number. Use a loop to determine if the target number exists in the list. Do not use the `in` operator.

```
# Example:  
# my_list = [10, 20, 30, 40, 50]  
# target = 30  
# Expected Output: 30 exists in the list.  
  
# target = 60  
# Expected Output: 60 does not exist in the list.
```

Question 55: Calculate the Average

Given a list of numbers, use a loop to calculate and print their average. You can use `len()` to get the count of elements, but avoid using `sum()` for the total.

Format the average to two decimal places.

```
# Example:  
# scores = [85, 90, 78, 92, 88]  
# Expected Output: The average score is: 86.60
```

Practice Questions: Looping Through Lists

Continue honing your looping skills with these challenges focusing on list manipulation and validation.

Question 56: Element-wise Sum of Two Lists

Given two lists of the same length, write Python code using a loop to create a new list where each element is the sum of the corresponding elements from both original lists.

```
# Example:  
# list1 = [10, 20, 30, 40]  
# list2 = [1, 2, 3, 4]  
# Expected Output: [11, 22, 33, 44]  
  
# list_a = [5, 15, 25]  
# list_b = [2, 4, 6]  
# Expected Output: [7, 19, 31]
```

Question 57: Check if a List is Sorted

Write a program that takes a list of numbers and, using a loop, determines whether it is sorted in ascending order. Print `True` if it is sorted, and `False` otherwise.

Do not use built-in `sort` or `sorted()` functions for checking.

```
# Example 1:  
# numbers = [1, 5, 10, 15, 20]  
# Expected Output: True  
  
# Example 2:  
# numbers = [1, 10, 5, 15, 20]  
# Expected Output: False  
  
# Example 3:  
# numbers = []  
# Expected Output: True (An empty list is considered sorted)
```

List Methods

List methods are built-in functions that operate directly on a list, allowing you to modify it in place. Unlike built-in functions like `sorted()` which return a new list, methods change the original list.

Adding Elements

`.append()` – Add to End

Adds a single element to the end of the list.

```
fruits = ["apple", "mango"]
fruits.append("banana")
print(fruits) # Output: ["apple", "mango", "banana"]
```

`.insert()` – Add at Index

Inserts an element at a specified index. The first argument is the index, the second is the value.

```
fruits = ["apple", "mango", "banana"]
fruits.insert(1, "kiwi")
print(fruits) # Output: ["apple", "kiwi", "mango", "banana"]
```

Removing Elements

`.remove()` – Remove by Value

Removes the first occurrence of a specified value from the list. If the item is not found, it raises a `ValueError`.

```
fruits = ["apple", "kiwi", "mango", "banana"]
fruits.remove("mango")
print(fruits) # Output: ["apple", "kiwi", "banana"]
```

`.pop()` – Remove by Index (or Last)

Removes and returns the element at a given index. If no index is specified, it removes and returns the last element.

```
fruits = ["apple", "kiwi", "banana"]
popped_last = fruits.pop()
print(f"Popped: {popped_last}, List: {fruits}") # Output: Popped: banana, List: ["apple", "kiwi"]

popped_first = fruits.pop(0)
print(f"Popped: {popped_first}, List: {fruits}") # Output: Popped: apple, List: ["kiwi"]
```

 Remember that list methods like `.append()`, `.insert()`, `.remove()`, and `.pop()` modify the list directly. If you need a modified version of the list while keeping the original intact, you'll often need to create a copy first.

List Methods: Sorting, Reversing, Searching & Counting

Beyond adding and removing elements, Python lists offer powerful methods for organising and querying their contents. These methods modify the list in place or provide information about its elements.

Sorting and Reversing Elements

`.sort()` - Sort In-Place

Sorts the elements of the list in ascending order directly, modifying the original list.

```
nums = [40, 10, 30, 20, 50]
nums.sort()           # sorts in place, modifies original
print(nums)           # Output: [10, 20, 30, 40, 50]
```

`.sort(reverse=True)` - Sort Descending

Sorts the list in descending order by setting the `reverse` parameter to `True`.

```
nums = [10, 20, 30, 40, 50] # Reset for example clarity
nums.sort(reverse=True)     # sorts in descending order
print(nums)                 # Output: [50, 40, 30, 20, 10]
```

`.reverse()` - Reverse In-Place

Reverses the order of elements in the list directly, modifying the original list.

```
nums = [10, 20, 30, 40, 50] # Reset for example clarity
nums.reverse()              # reverses in place
print(nums)                 # Output: [50, 40, 30, 20, 10]
```

Searching and Counting Elements

`.index()` - Find Element Index

Returns the index of the first occurrence of a specified value in the list.

```
fruits = ["apple", "mango", "banana", "mango"]
print(fruits.index("mango")) # Output: 1 — index of first occurrence
```

`.count()` - Count Element Occurrences

Returns the number of times a specified value appears in the list.

```
fruits = ["apple", "mango", "banana", "mango"]
print(fruits.count("mango")) # Output: 2 — how many times it appears
```

❗ It's crucial to remember that `.sort()` and `.reverse()` modify the list directly (in-place). If you need to preserve the original list, make a copy before applying these methods, or use built-in functions like `sorted()` (which returns a new sorted list).

List Methods: Copying and Clearing

Beyond adding, removing, sorting, and searching, Python lists provide methods for managing their overall state, including creating independent copies and completely clearing their contents.

`.copy()` - Create a Shallow Copy

Returns a new list that is a shallow copy of the original. This is crucial when you want to modify a list without affecting its source.

```
original_list = [10, 20, 30]
new_list = original_list.copy() # Creates an independent copy

new_list.append(40) # Modifies 'new_list'
print(original_list) # Output: [10, 20, 30] — 'original_list' remains
unchanged
print(new_list)      # Output: [10, 20, 30, 40]
```

`.clear()` - Remove All Elements

Removes all items from the list, effectively making it an empty list. This method modifies the list in place.

```
my_list = ["apple", "banana", "cherry"]
my_list.clear() # Removes all elements
print(my_list) # Output: []
```

- ❗ It's important to differentiate between methods that modify a list in-place (like `.sort()`, `.reverse()`, and `.clear()`) and functions that return a new modified list (like `sorted()` or creating a copy with `.copy()`). Always consider if you need to preserve the original list or if modifying it directly is acceptable.

Membership Operators

Membership operators are used to test if a sequence (like a list) contains a specified element. They return a Boolean value (True or False) based on the result of the check, making them highly useful for conditional logic.

in - Check for Membership

This operator returns True if the specified value is found within the list, and False otherwise.

```
fruits = ["apple", "banana", "cherry"]  
print("banana" in fruits) # Output: True  
print("grape" in fruits) # Output: False
```

not in - Check for Non-Membership

This operator returns True if the specified value is NOT found within the list, and False otherwise.

```
fruits = ["apple", "banana", "cherry"]  
print("grape" not in fruits) # Output: True  
print("banana" not in fruits) # Output: False
```

- ❗ Membership operators are fundamental for controlling program flow, allowing you to execute code blocks only when certain elements are present or absent from your lists.

Practice Questions: List Methods

Test your understanding of Python list methods and operators with these practice questions.

Q58. Find the largest and smallest number in a list without using built-in functions like `max()` or `min()`.

Hint: Use a loop and a variable to track the current largest/smallest as you go through the list. **Example:** if the list is `[3, 1, 4, 1, 5]`, the largest is 5 and smallest is 1.

Q59. Reverse a list without using the `.reverse()` method or list slicing (`[::-1]`).

Hint: Think about swapping elements from both ends of the list using a loop.

Example: `[1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]`

Q60. Given two lists, merge them into a single new list without modifying the originals.

Hint: Use the `+` operator or a loop to combine. **Example:** `list1 = [1, 2]`, `list2 = [3, 4]` → `merged = [1, 2, 3, 4]`

Practice Questions: List Methods

Challenge yourself with these problems to solidify your understanding of Python list manipulation and membership operators.

01

Question 61

Given a list, remove all duplicate elements while preserving the original order of the unique items.

```
# Example input:  
data = [10, 20, 30, 20, 10, 40, 50, 40]  
# Expected output: [10, 20, 30, 40, 50]
```

02

Question 62

Separate a list of integers into two distinct lists: one containing all the even numbers and the other containing all the odd numbers.

```
# Example input:  
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
# Expected output:  
# evens = [2, 4, 6, 8, 10]  
# odds = [1, 3, 5, 7, 9]
```

03

Question 63

Create a list containing the squares of numbers from 1 to 10 (i.e., [1, 4, 9, ..., 100]).

```
# Expected output:  
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

📌 These questions encourage you to think about iteration, conditional logic, and efficient list manipulation. Remember to consider edge cases!

Practice Questions: List Methods

Further challenge your understanding of list manipulation with these focused practice problems.

1

Question 64

Given a list of numbers (which may contain duplicates), write a Python script that takes an integer as input from the user and removes all occurrences of that integer from the list.

```
# Example input list:  
my_list = [10, 20, 10, 30, 20, 10, 40]  
# If user enters 10, expected output: [20, 30, 20, 40]
```

2

Question 65

Write a Python script that iterates through a list of integers and replaces every negative number found in the list with the value 0.

```
# Example input list:  
numbers = [5, -3, 8, -1, 0, -10, 12]  
# Expected output: [5, 0, 8, 0, 0, 0, 12]
```

List Comprehension

List comprehension offers a concise and readable way to create new lists from existing sequences or ranges. It condenses common `for` loop patterns (iteration + appending) into a single, elegant line of code, enhancing efficiency and clarity.

Basic Usage: For Loop vs. One Line

Transforming a loop that builds a list into a single expression.

```
# Normal way
squares = []
for i in range(1, 6):
    squares.append(i * i)
print(squares)  # [1, 4, 9, 16, 25]

# List comprehension — same result, one line
squares = [i * i for i in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

Conditional Filtering

Include elements in the new list only if they meet a specific condition.

```
# With a condition — only even numbers
evens = [i for i in range(1, 21) if i % 2 == 0]
print(evens)  # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# From an existing list — filter marks above 80
marks = [85, 60, 92, 45, 78, 95, 50]
passed = [m for m in marks if m > 80]
print(passed)  # [85, 92, 95]
```

Transforming Elements

Apply a function or operation to each item as it's added to the new list.

```
names = ["rahul", "priya", "karan"]
upper = [name.upper() for name in names]
print(upper)  # ["RAHUL", "PRIYA", "KARAN"]
```

Quick Tip: List comprehension excels for simple transformations and filters. If the logic becomes overly complex or involves multiple nested loops and conditions, consider reverting to a traditional `for` loop for better readability. Prioritise clarity over being overly clever.

Practice Questions: List Comprehension

Put your list comprehension skills to the test with these challenges!

1

Question 66

Using list comprehension, create a list of squares of all odd numbers from 1 to 20.

```
# Expected output:  
# [1, 9, 25, 49, 81, 121, 169, 225, 289, 361]
```

2

Question 67

Given a list of marks, use list comprehension to create a new list that contains only the marks that are above 75.

```
# Example input:  
marks = [65, 80, 70, 90, 75, 82, 60]  
# Expected output:  
# [80, 90, 82]
```

③ Remember, list comprehensions are about writing concise and efficient code. Think about how to combine iteration, conditions, and transformations in a single line.

Nested Lists

A nested list is essentially a list that contains other lists as its elements. This powerful structure allows us to represent complex data, such as **grids, tables, or matrices**, within Python.

Representing a Matrix

Consider a simple 3x3 matrix. Each inner list represents a row in the matrix:

```
# A 3x3 matrix
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Accessing Elements

To access elements in a nested list, you use multiple indexers: the first index specifies the inner list (row), and the second specifies the element within that inner list (column).

```
# Accessing elements — matrix[row][column]
print(matrix[0])    # Output: [1, 2, 3]    — first row
print(matrix[1][2]) # Output: 6           — row 1, column 2
print(matrix[2][0]) # Output: 7           — row 2, column 0
```

Practice Questions: Nested Lists

Challenge your understanding of nested lists by tackling these problems, focusing on element access and manipulation.

1

Question 68

Create a 3x3 matrix (a nested list) and then use nested loops to calculate and print the sum of all its elements.

```
# Example:
# matrix = [
#   [1, 2, 3],
#   [4, 5, 6],
#   [7, 8, 9]
# ]
# Expected output for this example: 45
```

2

Question 69

Given a 3x3 matrix as input, print its lower triangle. Replace all elements in the upper triangle (above the main diagonal) with an asterisk (*).

```
# Input:
# 1 2 3
# 4 5 6
# 7 8 9

# Expected Output:
# 1 * *
# 4 5 *
# 7 8 9
```

Practice Questions: Nested Lists

Further challenge your understanding of nested lists by tackling these problems, focusing on element access and manipulation.

Question 70

Given a 3x3 matrix, print its upper triangle. Replace all elements in the lower triangle (below the main diagonal) with an asterisk (*).

Input:

1 2 3

4 5 6

7 8 9

Expected Output:

1 2 3

* 5 6

* * 9

Question 71

Given a 3x3 matrix, print only the main diagonal elements and replace everything else with an asterisk (*).

Input:

1 2 3

4 5 6

7 8 9

Expected Output:

1 * *

* 5 *

* * 9

Practice Questions: Nested Lists

Dive deeper into nested list manipulation with these challenging problems, focusing on specific element identification.

Question 72

Given a 3x3 matrix, print only the anti-diagonal (top-right to bottom-left) elements and replace everything else with an asterisk (*).

```
# Input:
```

```
# 1 2 3
```

```
# 4 5 6
```

```
# 7 8 9
```

```
# Expected Output:
```

```
# * * 3
```

```
# * 5 *
```

```
# 7 * *
```

Question 73

Given a 4x4 matrix, print only the border elements and replace the inner elements with an asterisk (*).

```
# Input:
```

```
# 1 2 3 4
```

```
# 5 6 7 8
```

```
# 9 10 11 12
```

```
# 13 14 15 16
```

```
# Expected Output:
```

```
# 1 2 3 4
```

```
# 5 * * 8
```

```
# 9 * * 12
```

```
# 13 14 15 16
```